

- [Home \(/\)](#)
- [ASP.NET Core \(/asp-net-core\)](#)
- [Tutorials \(/razor-pages/tutorial/bakery7\)](#)
- [Razor Page Files \(/razor-pages\)](#)
- [Razor Syntax \(/razor-syntax\)](#)
- [Page Models \(/razor-pages/pagemodel\)](#)
- [Tag Helpers \(/razor-pages/tag-helpers/\)](#)
- [View Components \(/razor-pages/view-components\)](#)
- [Routing and URLs \(/razor-pages/routing\)](#)
- [Application Startup \(/application-configuration\)](#)
- [Configuration \(/configuration\)](#)
- [Middleware \(/middleware\)](#)
- [Dependency Injection \(/advanced/dependency-injection\)](#)
- [Working With Forms \(/razor-pages/forms\)](#)
- [Validation \(/razor-pages/validation\)](#)
- [Model Binding \(/razor-pages/model-binding\)](#)
- [State Management \(/razor-pages/state-management\)](#)
- [Caching \(/razor-pages/caching\)](#)
- [Managing Security With ASP.NET Identity \(/identity\)](#)
- [Using AJAX \(/razor-pages/ajax\)](#)
 - [Posting Forms \(/razor-pages/ajax/form-post\)](#)
 - [Partial Page Updates \(/razor-pages/ajax/partial-update\)](#)
 - [Cascading Dropdowns \(/razor-pages/ajax/cascading-dropdowns\)](#)
 - [Uploading Files \(/razor-pages/ajax/upload-files\)](#)
 - [Unobtrusive AJAX \(/razor-pages/ajax/unobtrusive-ajax\)](#)
- [Working with JSON \(/web-api\)](#)
- [Scaffolding \(/miscellaneous/scaffolding\)](#)
- [Publishing To IIS \(/publishing/publish-to-iis\)](#)
- [Advanced \(/advanced\)](#)
- [Table Of Contents \(/table-of-contents\)](#)

Cascading Dropdowns With AJAX in Razor Pages

The cascading dropdown pattern is typically used to help users to filter data. The first dropdown is populated with the broadest options, and subsequent dropdowns are populated with options that relate to the selected value of the preceding dropdown. This requirement does not need AJAX, but using an approach that incorporates AJAX requests provides a much better experience for the user.

In the
be ob
SubCa



LANDERS

SUPER CRAZY SALE

PISO SALE • BUY 1 GET 1 • 50% OFF

MARCH
ONLINE & IN

will
and
D

```

public interface ICategoryService
{
    IEnumerable<Category> GetCategories();
    IEnumerable<SubCategory> GetSubCategories(int categoryId);
}
public class CategoryService : ICategoryService
{
    public IEnumerable<Category> GetCategories()
    {
        return new List<Category>
        {
            new Category { CategoryId = 1, CategoryName="Category 1" },
            new Category { CategoryId = 2, CategoryName="Category 2" },
            new Category { CategoryId = 3, CategoryName="Category 3" }
        };
    }
    public IEnumerable<SubCategory> GetSubCategories(int categoryId)
    {
        var subCategories = new List<SubCategory> {
            new SubCategory { SubCategoryId = 1, CategoryId = 1, SubCategoryName="SubCategory 1" },
            new SubCategory { SubCategoryId = 2, CategoryId = 2, SubCategoryName="SubCategory 2" },
            new SubCategory { SubCategoryId = 3, CategoryId = 3, SubCategoryName="SubCategory 3" },
            new SubCategory { SubCategoryId = 4, CategoryId = 1, SubCategoryName="SubCategory 4" },
            new SubCategory { SubCategoryId = 5, CategoryId = 2, SubCategoryName="SubCategory 5" },
            new SubCategory { SubCategoryId = 6, CategoryId = 3, SubCategoryName="SubCategory 6" },
            new SubCategory { SubCategoryId = 7, CategoryId = 1, SubCategoryName="SubCategory 7" },
            new SubCategory { SubCategoryId = 8, CategoryId = 2, SubCategoryName="SubCategory 8" },
            new SubCategory { SubCategoryId = 9, CategoryId = 3, SubCategoryName="SubCategory 9" }
        };
        return subCategories.Where(s => s.CategoryId == categoryId);
    }
}

```

Start Chatting 1:1 in Minutes

Connect 1-on-1 with an Expert for Help and Get Your Answer in Minutes, 24/7.



The `GetCategories` method returns all `Category` instances. The `GetSubCategories` method returns a collection of `SubCategory` entities, filtered by the `categoryId` value passed in. The interface and its implementation need to be registered with the dependency injection system (/advanced/dependency-injection) in `Startup`:

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<ICategoryService, CategoryService>();
}

```

The `PageModel` class for the Razor page where the dropdowns will appear is as follows:



```

public class CascadingDropdownsModel : PageModel
{
    private ICategoryService categoryService;
    public CascadingDropdownsModel(ICategoryService categoryService) => this.categoryService = categoryService;
    [BindProperty(SupportsGet = true)]
    public int CategoryId { get; set; }
    public int SubCategoryId { get; set; }
    public SelectList Categories { get; set; }

    public void OnGet()
    {
        Categories = new SelectList(categoryService.GetCategories(), nameof(Category.CategoryId), nameof(Category.CategoryName));
    }
    public JsonResult OnGetSubCategories()
    {
        return new JsonResult(categoryService.GetSubCategories(CategoryId));
    }
}

```

The `CategoryService` is injected into the constructor and assigned to a private field for later use. Three public properties have been added to the `PageModel`. The first is an `int` representing the selected `CategoryId` value. The AJAX request will use the `GET` verb, so the `SupportsGet = true` option has been applied to the `BindProperty` attribute that decorates the property. The second property represents the selected `SubCategory`. Its only use in this example is to render a [select tag helper \(/razor-pages/tag-helpers/select-tag-helper\)](#), but if you want to capture the selected value using model binding, you would need to decorate the property with the `BindProperty` attribute accordingly.

The final public property is a `SelectList` type, and will represent the options for the initial Category dropdown list. The property is instantiated in the `OnGet` handler method.

Another handler method `OnGetSubCategories`, this one being a [named handler method \(/razor-pages/handler-methods#named-handler-methods\)](#), is added. It returns the filtered `SubCategory` collection as JSON.

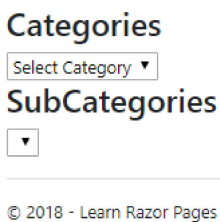
The content page contains two select tag helpers:

```

<h4>Categories</h4>
<select asp-for="CategoryId" asp-items="Model.Categories">
    <option value="">Select Category</option>
</select>
<h4>SubCategories</h4>
<select asp-for="SubCategoryId"></select>

```

The first is populated with the categories while the second is initially empty:



The content page also contains a script block that utilises jQuery to add an event listener to the first drop down and make the AJAX call to the named handler:

```

@section scripts{
<script>
    $(function () {
        $("#CategoryId").on("change", function() {
            var categoryId = $(this).val();
            $("#SubCategoryId").empty();
            $("#SubCategoryId").append("<option value=''>Select SubCategory</option>");
            $.getJSON('?handler=SubCategories&categoryId=' + categoryId, (data) => {
                $.each(data, function (i, item) {
                    $("#SubCategoryId").append("<option value='" + item.subCategoryId + "'>" + item.subCategoryName + "</option>");
                });
            });
        });
    });
</script>
}

```

Discover more	
Software framework	>
Application programming interface	>
Web framework	>

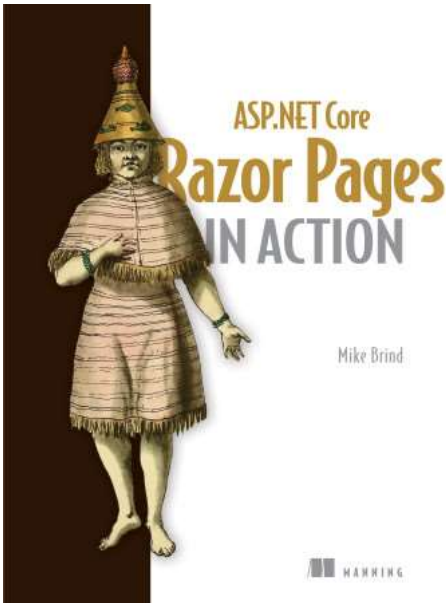
Each time the selection in the first dropdown is changed, the second one is cleared, and then populated with the SubCategories returned from the `getJSON` call.

Fetch [↗](#)

The following code block is the equivalent to the previous one except that it uses the Fetch API and no jQuery at all:

```
@section scripts{
<script>
    document.getElementById('CategoryId').addEventListener('change', (e) => {
        document.getElementById('SubCategoryId').innerHTML = "<option value=''>Select SubCategory</option>";
        fetch(`?handler=SubCategories&categoryId=${e.target.value}`)
            .then((response) => {
                return response.json();
            })
            .then((data) => {
                Array.prototype.forEach.call(data, function (item, i) {
                    document.getElementById('SubCategoryId').innerHTML += `<option value="${item.subCategoryId}">${item.subCategoryName}</option>`;
                });
            });
    });
</script>
}
```

Last updated: 8/15/2019 9:49:14 AM



[https://www.manning.com/books/asp-net-core-razor-pages-in-action?](https://www.manning.com/books/asp-net-core-razor-pages-in-action?utm_source=mikebrind&utm_medium=affiliate&utm_campaign=book_brind_razor_7_26_21&a_aid=mikebrind&a_bid=f71bcc8c)

[utm_source=mikebrind&utm_medium=affiliate&utm_campaign=book_brind_razor_7_26_21&a_aid=mikebrind&a_bid=f71bcc8c\)](https://www.manning.com/books/asp-net-core-razor-pages-in-action?utm_source=mikebrind&utm_medium=affiliate&utm_campaign=book_brind_razor_7_26_21&a_aid=mikebrind&a_bid=f71bcc8c)





On this page

Cascading Dropdowns With AJAX in Razor Pages

Fetch

Discover more

[Microsoft, Razor](#)



[APIs](#)



[development](#)



[Visual Studio](#)



[frameworks](#)



[Web framework](#)



[Software](#)



[.NET Core](#)



Latest Updates

Working With Data (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/working-with-data>)

View Components in Razor Pages (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/view-components>)

Adding TypeScript Support to Razor Pages (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/typescript>)

Scaffolding CRUD pages (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/scaffolding>)

Adding Sass Support (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/sass>)

Adding a New Page (<https://www.learnrazorpages.com/razor-pages/tutorial/bakery7/new-page>)

© 2018 - 2026 - Mike Brind.
All rights reserved.
Contact me at Mike dot Brind at Outlook.com

Discover more [API](#) [Razor](#) [NET Core](#) [development](#) [web development framework](#) [framework](#) [APIs](#) [Sof](#)
[Visual Studio](#) [Microsoft, Razor](#)

